A Live-and-Safe Finalizer-Progress Design

Response to Stall-Loop-Design §5.10.1 (Finalizer keeps progress)

F1R3node Casper Working Group

October 16, 2026

Abstract

Stall-loop-design §5.10.1 proposes that, on a deadline, the finalizer "adopts the highest candidate proven so far." That rule is neither safe nor live. We identify three concrete safety counterexamples in the current Rust implementation and show that the proposal has no progress argument at all when the first full certification never completes. We then present a design that separates three concerns – resumable *Compute*, pre-commit *Validate*, and atomic *Commit* – preserving work across deadlines while discharging every safety condition under the snapshot observed at commit time. A verifiable replacement for §5.10.1 and an implementation sketch against the existing codebase are included. Related work tracking: F1R3FLY-io/f1r3node-rust issue #24.

Contents

1 Context **1**

2 Why §5.10.1 is neither safe nor live as written **1**
- 2.1 Safety counterexample 1: incompatibility bypass under $\theta \leq 0$ . . . 1
- 2.2 Safety counterexample 2: state-containment bypass . . . 2
- 2.3 Safety counterexample 3: TOCTOU between Advance and commit . . . 2
- 2.4 Liveness insufficiency: no progress argument . . . 2

3 Design: resumable derivation with atomic validated commit **2**
- 3.1 Separation of three concerns . . . 2
- 3.2 Resumable floor derivation . . . 3
- 3.3 Snapshot identity σ . . . 3
- 3.4 Snapshot-change policy . . . 4
- 3.5 Atomic validated commit . . . 4
- 3.6 Idempotent event publication . . . 5
- 3.7 Liveness argument . . . 5

4 Replacement text for §5.10.1 **6**

5 Verification requirements **6**

6 Implementation sketch **7**

7 Open questions **8**

1 Context

Issue #24 reports a stall pathology in which validators eventually stop advancing the Last Finalized Block (LFB). The current implementation wraps `compute_last_finalized_block` in a 15 s `tokio::time::timeout` (`casper/src/rust/engine/multi_parent_casper/finalization_runner.rs:278`). On expiry it drops the entire future, discards every memo built in the run, and reschedules. If one derivation exceeds the budget, subsequent runs inherit no accumulated work and the node can stall indefinitely while remaining responsive on every other interface.

§5.10.1 proposes to adopt the highest candidate "proven so far" on timeout and to memoize oracle verdicts per (target, snapshot). The underlying insight is sound: discarding every cycle's work is wasteful. The specific rule, however, conflates three distinct concerns:

(i) *finding a certified candidate*,

(ii) *computing a safe finality decision*, and

(iii) *guaranteeing eventual progress*.

Collapsing them into one rule produces unsafety on (ii) and silence on (iii).

2 Why §5.10.1 is neither safe nor live as written

2.1 Safety counterexample 1: incompatibility bypass under $\theta \leq 0$

`FloorOfView` in `casper/src/rust/finality/floor.rs:405-440` explicitly recognises that, when the fault-tolerance threshold is non-positive, two disjoint half-stake cliques can certify incompatible same-height blocks without equivocating. A complete derivation returns `IncompatibilityHold`; the surrounding `hold_for` (`floor.rs:461`) explicitly converts `CasperError::IncompatibleFinalizedFork` to a hold when $\theta_{\text{num}} \leq 0$. Under $\theta > 0$, the same error remains a loud safety alarm.

Let A and B be conflicting blocks and let the committee split evenly. Suppose a validator evaluates A first, witnesses the half-stake certificate, and hits the deadline before evaluating B. §5.10.1 would finalize A. A peer that evaluates B first finalizes B. The full derivation would have detected the split and refused both.

The lesson: a candidate-level oracle verdict is not a finality decision. Under $\theta \leq 0$ the sibling scan is load-bearing for safety and must not be bypassed by a deadline.

2.2 Safety counterexample 2: state-containment bypass

Even under $\theta > 0$, an oracle certificate alone is insufficient. `floor_of_view_metered` at `floor.rs:619-642` computes a candidate floor and then verifies, via `state_contains_metered`, that the derived floor contains every effect settled under the current LFB. On failure, it returns `ContainmentHold`, not `Advance`. A truncated run that stops immediately after the oracle verdict but before the containment check would adopt a floor that regresses settled state. That is a monotonicity violation of exactly the kind `ContainmentHold` exists to prevent.

2.3 Safety counterexample 3: TOCTOU between Advance and commit

A race already exists in the current code, independent of §5.10.1, which §5.10.1 would widen. Between `floor_of_view(&dag, ...)` returning `Advance(F)` at `finalization_runner.rs:504` and `record_directly_finalized` taking the metadata write lock inside `block-storage/src/rust/dag/block_dag_key`

new latest-messages can be admitted. The admitted state may invalidate F: a validator that previously carried F's majority may advertise a different latest message, or a newly-held block may introduce effects that F's state does not contain. The current code only checks `directly_finalized_hash` $\in$ `dag_set`; it does not re-check `state_contains(F,` LFB_{cur}`)`.

2.4 Liveness insufficiency: no progress argument

§5.10.1 addresses only the case in which some candidate is already proven when the deadline fires. If every run truncates before producing one certified candidate, adopting "the highest proven so far" is vacuous. Within-run memoization does nothing when nothing was proved; across-run memoization is unsafe without a precise snapshot-change policy. The design document does not state the synchrony and fairness assumptions a progress argument would require, so no such argument can be constructed from the current text.

3 Design: resumable derivation with atomic validated commit

3.1 Separation of three concerns

We separate the finalizer loop into three phases with disjoint invariants.

Compute
: A resumable derivation state machine with a bounded budget. Produces either `Yielded(cursor)` (budget exhausted mid-run) or `Terminal(FloorOfView,` σ`)` (derivation complete under some snapshot σ). Never commits.

Validate
: Re-runs the safety checks that depend on the *current* snapshot σ' while holding the metadata write lock. Specifically re-checks state-containment, the ftt-regime incompatibility rule, and the canonical-order (A-LIN) invariant for candidate F against LFB_{cur} under σ'.

Commit
: One atomic LMDB `write_txn` across block metadata, a new finalization commit marker, and the LFB pointer register. On success, publishes events in deterministic order; on crash, startup reconciliation replays via the commit marker.

A deadline fires between Compute steps only. It never cancels Validate or Commit.

3.2 Resumable floor derivation

The derivation is explicit about its state. We add `FloorCursor` next to `FloorOfView`:

```
enum FloorCursor {
  PendingTips        { sigma, live_snapshot },
  PendingDecidable   { sigma, tips, live_snapshot, missing },
  PendingCandidates  { sigma, tips_scope, pivot, live_snapshot },
  PendingCertificate { sigma, candidates, next_idx, ... },
  PendingParentFloors { sigma, candidates, parents, parent_idx,
                        inherited_so_far },
  PendingCompatibility { sigma, ordered, inherited, cand_idx,
                         inh_idx, memo },
  PendingContainment { sigma, derived, current, memo },
```

```
  Ready(FloorOfView, SnapshotId),
}
```

The step function has signature

$$\texttt{step_floor_derivation} : (\texttt{FloorCursor}, \texttt{Budget}, \texttt{Ctx}) \to \texttt{StepOutcome}$$

where `StepOutcome` is `Yielded(FloorCursor)` or `Terminal(FloorOfView, SnapshotId)`. The budget is checked only at step boundaries, never inside a sub-computation; a `WorkMeter` step-count budget is used in tests for determinism, wall-clock in production.

Yield points coincide with phase boundaries already present in `floor_of_view_metered` (`floor.rs:526-643`): after tips enumeration, between candidates in the oracle sweep (one `CliqueOracle::ft_witnessed_exact_metered` call is one step), per parent in `parent_frontier_metered`, per `(cand, other)` pair in the compatibility loop, and `state_contains_metered` as one step. No helper requires sub-step surgery for a usable first version.

3.3 Snapshot identity σ

σ is a 32-byte Blake2b hash whose preimage, in canonical order, is:

1. Sorted (validator_id, latest_message_hash) pairs from `dag.latest_message_hashes()`.
2. Current LFB (hash, block_number). Bonds and epoch are pure functions of the LFB, so folding the LFB hash captures them transitively without an explicit entry.
3. The threshold scalar θ_{num}:θ_{den}.
4. Blake2b of the equivocation-tracker state consulted by `dag.own_testimony_metered`.

σ is computed once at step entry and recorded in the cursor's state.

3.4 Snapshot-change policy

Not every memo depends on σ. The policy is explicit about which cached work survives a snapshot change and which does not.

Survives σ change
: Per-block memos keyed on immutable block hashes: `frontier_index` and `ft_lower_bound` on `BlockMetadata`; lineage `IntroducedSigsMemo` entries keyed on `(child_hash, ancestor_hash)`. These are pure functions of their keys.

Discarded on σ change
: Everything that read `live_snapshot`, tips, or candidate selection: the entire cursor is reset to `PendingTips` under the new σ.

Per-block memos remain warm; the discarded work is bounded by $O(|\text{frontier}|)$ per σ change, not $O(|\text{DAG}|)$.

Anti-livelock. If σ invalidates N consecutive runs (configurable, default 4), the finalizer freezes to the last stable σ for one run. The resulting `Advance(F)` is still safe because Validate re-checks containment under current σ' at commit time; a stale F is rejected and the next cycle re-derives.

3.5 Atomic validated commit

The commit path is hoisted into

`commit_lfb`(directly, indirect_sorted, ft_value) → `Result<CommitReceipt, KvStoreError>`

with explicit preconditions discharged while holding `global_lock.write`:

P1. directly ∈ dag_set.

P2. `state_contains(directly,` LFB_{cur}`)` – closes Counterexample 2.3.

P3. Under $\theta > 0$, no `IncompatibleFinalizedFork` on candidate's siblings in the current snapshot (loud alarm, not hold); under $\theta \leq 0$, `IncompatibilityHold` is a terminal halt – closes Counterexample 2.1.

P4. A-LIN assertion: `directly` is a main-parent-chain descendant of LFB_{cur}. On violation, halt with `KvStoreError::ALinViolation`. (Converts an unproven assumption flagged as Q-1 in the design document into an observable safety alarm.)

If any precondition fails, the commit aborts with `KvStoreError::StateChanged(hash)` and the runner re-derives in the next cycle. A single bounded retry is permitted; the historical 128-retry outer loop is removed as it hides concurrency bugs rather than tolerating them.

One LMDB transaction. The write uses `batched_put` at `shared/src/rust/store/lmdb_key_value_store.rs` across three stores sharing one `Env`:

- `block_metadata` – updated `BlockMetadata` rows for directly- and indirectly-finalized blocks.
- `finalization_commit_marker` – new store, key = directly-finalized `BlockHash`, value = {directly, indirect_sorted, ft_value, height, commit_epoch, events_published}.
- `lfb_pointer` – new one-row register, overwrite with (hash, height, commit_epoch).

LMDB serialises writers on one env-wide lock, so this is one atomic, durable transaction. The `DagState` in-memory swap uses `imbl`'s copy-on-write primitives *after* the transaction commits, under the same write guard. On crash before the swap, the next `try_new` reads from KV, which is authoritative.

3.6 Idempotent event publication

Events publish *after* the LMDB commit returns `Ok`, in sorted (block_number, block_hash) order; today the code iterates a `HashSet` non-deterministically at `finalization_runner.rs:402-466`.

A new `BlockFinalisedV2` variant in `shared/src/rust/shared/f1r3fly_event.rs:98-113` carries commit_epoch. The marker is written in the commit batch; on startup `reconcile_pending_finalization` (`block_metadata_store.rs:104`) reads the marker:

Observed state	Action
marker present, `events_published = true`	no-op
marker present, `events_published = false`	re-emit `BlockFinalisedV2` in sorted order; flip flag
marker absent, no stale `directly_finalized` flag	no-op
marker absent, stale metadata flag present	`panic` (corruption: impossible under atomic path; assertion catches regressions)

Replays carry the same commit_epoch, so dedup-aware consumers treat re-emits as the same logical event. Current consumers are either already idempotent (block-report prewarm) or forward the field for client dedup.

Wire-compat note: `bincode` is not trailing-field tolerant. Adding a new enum variant is correct; extending the existing struct is not.

3.7 Liveness argument

Under the following assumptions:

- eventual synchrony (bounded message delay post-GST),
- honest-stake supermajority appropriate to the ftt regime ($\geq 2/3$ under $\theta > 0$),
- fair block propagation,
- a per-run budget sufficient to complete at least one derivation step,

the design guarantees eventual commit:

L1. Per-block immutable memos accumulate monotonically across runs and are never invalidated.

L2. Post-GST, σ eventually stabilises for one cycle's worth of work (follows from eventual synchrony and bounded latest-message rate).

L3. Under stable σ, the cursor transitions `PendingTips` $\rightarrow$ `PendingDecidable` $\rightarrow \ldots \rightarrow$ `Ready` in $O(|\text{frontier}|)$ steps.

L4. Validate under current σ' is a cheap single-step check; worst case, we restart under σ' and L1 keeps all per-block work.

L5. Commit is atomic with respect to crash and concurrent runs.

Progress is strictly monotone along σ-stable windows. The anti-livelock rule in §3.4 ensures that pathological latest-message churn cannot forever prevent a stable window being offered to Compute; Validate still gates safety under σ'.

4 Replacement text for §5.10.1

5.10.1 Finalizer forward progress. *The finalizer performs incremental, resumable derivation of the live-view floor over a snapshot identity σ computed from the latest-message set, the current LFB hash, the fault-tolerance threshold, and the equivocation-evidence hash. A per-run budget bounds the search; a derivation step either advances a cursor $k \rightarrow k{+}1$ atomically or yields its remaining budget without producing a decision. Expiration of the budget preserves the cursor as a resumption token. A change in σ resets the cursor but preserves per-block immutable memos (frontier index, FT lower bound, lineage sig-introduction memos).*

A candidate F is adopted iff the cursor reaches Ready*(*FloorOfView*::*Advance*(F))* *under some σ* and *a re-validation under the current σ' – holding the metadata write lock – confirms state-containment against the current LFB, candidate soundness against inherited parent floors, the ftt-regime incompatibility rule, and the canonical-order (A-LIN) invariant. Adoption writes block-metadata flags, a finalization commit marker, and the LFB pointer in a single atomic LMDB transaction. Event publication occurs after*

the transaction commits; the commit marker's events_published *flag enables idempotent replay under crash recovery.*

Memoization uses two layers: per-block immutable resolutions (survive σ changes) and (target, σ)-keyed oracle verdicts (invalidated on σ change). Under eventual synchrony, honest-stake assumptions appropriate to the ftt regime, fair propagation, and a budget sufficient for at least one derivation step per run, the cursor advances strictly monotonically along σ-stable windows and finalization completes in a bounded number of resumptions.

5 Verification requirements

We propose the following mandatory regression suite before accepting A1, in addition to the existing 60-hour soak (which is operational evidence, not a correctness substitute).

Property	**Test**
No conflicting finality	Two nodes with reversed candidate orderings on an evenly split committee; under $\theta > 0$ both reject with `IncompatibleFinalizedFork`; under $\theta \leq 0$ both return `IncompatibilityHold`.
No settled-state regression	Oracle-certified candidate whose state omits an LFB-settled effect $\Rightarrow$ commit aborts with `StateChanged`; cursor preserved.
Incomplete-proof rejection	Injected deadline at every `FloorCursor` variant $\Rightarrow$ `Ready` never reached from truncation; `commit_lfb` never called.
Cache correctness	Property-based: for fixed σ, cached and uncached derivations yield identical `FloorOfView`.
Resumable liveness	Deterministic `WorkMeter` budget forces ≥ 10 resumptions on a 50-validator net; cursor reaches `Terminal(Advance(_))`.
Atomic adoption	Fault-point between marker write and event publish; restart causes replay with same `commit_epoch`; consumer dedup verifies exactly-once logical delivery.
Canonical-order safety	Synthetic DAG where candidate is not on LFB_{cur}'s main-parent chain $\Rightarrow$ `commit_lfb` halts with `ALinViolation`.
TOCTOU re-validation	Inject new latest-message between `Ready(Advance(F))` and `commit_lfb` write lock; re-validation rejects; next cycle re-derives.
Operational soak	Instrumented 60-hour run with `finalizer_cursor_yields_total`, `_resumes_total`, `_invalidations_total`, `commit_lfb_toctou_rejections_total` monotonically bounded and LFB advancing.

6 Implementation sketch

The design can land as a focused change set that avoids rewriting `floor.rs`:

File	Change
casper/.../finality/floor_runner.rs *(new)*	FloorCursor, StepOutcome, SnapshotId, Budget, step_floor_derivation.
casper/.../finality/floor.rs :405-440, 526-643, 918-1051	Expose helpers pub(crate); extract choose_compatible_step; existing floor_of_view_metered becomes thin wrapper of the runner.
casper/.../multi_parent_casper/finalization_runner.rs :255-295, 377-487, 570-614	Drop tokio::time::timeout; drive cursor via step_floor_derivation; split commit from compute.
block-storage/.../block_dag_key_value_storage.rs :1575-1697	Rewrite record_directly_finalized as commit_lfb; move ancestry walk inside write lock; drop 128-retry loop; add pre-commit revalidation.
block-storage/.../block_metadata_store.rs :104-119, 213-270	Split prepare_finalized_batch from apply_finalized_in_memory; add reconcile_pending_finalization on startup.
block-storage/.../finalization_commit_marker.rs *(new)*	Commit marker store + schema.
shared/.../f1r3fly_event.rs:98-113	Add BlockFinalisedV2 { commit_epoch, ...} variant.
block-storage/.../soak_snapshot.rs:129, 869-957	Include commit marker and LFB pointer in snapshot/restore.

Pattern references already in-tree (reuse, don't reinvent): WorkMeter abstraction threaded through floor.rs; imbl::OrdMap/HashSet copy-on-write in BlockMetadataStore for the DagState swap; batched_put at shared/.../lmdb_key_value_store.rs:183 for cross-store atomicity; FinalityProgress struct at casper/.../block_creator.rs as a progress-tracking pattern.

7 Open questions

Q1. Cursor durability across restarts. We propose in-memory only for v1; fresh PendingTips is cheap and a persistence schema would need its own versioning. Revisit if restarts during long derivations are observed.

Q2. propagate_ft_to_finalized_blocks boundary. Today runs after persistence at block_dag_key_value_stora raises a monotonic lower bound, so partial crashes self-heal. Likely acceptable to leave outside the atomic commit; worth stating explicitly in the final design.

Q3. commit_epoch semantics. Per-LFB (incremented at every commit; marker overhead = one u64) gives tightest dedup; per-boot is coarser. We recommend per-LFB.

Q4. A-LIN assumption Q-1. The design document flags A-LIN as open. The proposed explicit halt converts an unprovable assumption into an observable safety alarm, which is strictly better than silent reliance. A separate correctness obligation remains: either prove A-LIN under the v6 ordering rules, or document the halt as the canonical response.

Summary

§5.10.1's performance intuition is correct and worth keeping: avoid discarding reusable work on every cycle. Its specific rule, however, promotes an incomplete oracle verdict to a finality decision,

which violates both $\theta \leq 0$ incompatibility handling and state containment, and widens an existing TOCTOU race at commit. The replacement separates *Compute* (resumable, budgeted), *Validate* (safety checks under current snapshot), and *Commit* (atomic, idempotent), with a precise snapshot-identity and change policy that makes memoization safe. Liveness becomes a conditional claim under explicit synchrony and fairness assumptions, with per-block memos as the monotone progress carrier. The design maps directly onto the existing F1R3node implementation and can land as a focused change set backed by a mandatory regression suite.